Error handling in Go

Go has no exceptions. Errors are values returned alongside results. The handling pattern is “check it, decide what to do, move on.” Here’s how that works in practice, the libraries that support it, and the patterns that show up in production code.

Errors vs exceptions: the Go philosophy

Most languages throw exceptions that unwind the stack until something catches them. Go returns errors as ordinary values from functions. The caller decides what to do.

	Exceptions (Java, Python, C++)	**Go errors**
Mechanism	Stack-unwinding throw/catch	Value returned from function
Visible in signature	Sometimes (checked exceptions)	Always (`error` return)
Cost	High (stack unwind)	Low (one comparison)
Easy to ignore	Hard (compiler enforces or unwinds)	Yes (silently `_`)
Best for	Truly exceptional, unrecoverable	Every failure path

Go does have `panic` / `recover`, which behaves like a stack-unwinding exception. Reserve it for actual bugs. Use returned errors for everything else.

The error interface

```
type error interface {
    Error() string
}
```

That’s it. Anything with an `Error() string` method satisfies `error`. The interface lives in the builtin package, so you never import it.

A nil `error` means success. A non-nil error means something went wrong.

```go
f, err := os.Open("file.txt")
if err != nil {
    return err
}
defer f.Close()
```

The convention is: error is the last return value, and you check it immediately.

Creating errors

errors.New

Simplest form: a string error.

```go
import "errors"

err := errors.New("not found")
```

`errors.New` returns a pointer to an `errorString` struct. Two `errors.New("x")` calls return distinct values, even if the message matches. This matters: see Sentinel errors below.

fmt.Errorf

Formatted, with optional wrapping.

```go
err := fmt.Errorf("opening %s: %w", path, baseErr)
```

The `%w` verb wraps `baseErr`. The resulting error has `baseErr` in its chain (accessible via `errors.Is` / `errors.As`).

Without `%w`, use `%v` or `%s` for plain interpolation; the underlying error becomes part of the message but isn't recoverable.

```go
err := fmt.Errorf("opening %s: %v", path, baseErr)   // message only
err := fmt.Errorf("opening %s: %w", path, baseErr)   // wrapped
```

Use `%w` unless you have a specific reason not to.

Sentinel errors

A package-level error value used for exact comparison.

```go
package fs

var ErrNotFound = errors.New("file not found")

func Read(name string) ([]byte, error) {
    if !exists(name) {
        return nil, ErrNotFound
    }
    // ...
}
```

Callers check with `errors.Is`:

```go
data, err := fs.Read("x.txt")
if errors.Is(err, fs.ErrNotFound) {
    // handle missing file
}
```

Examples in the standard library: `io.EOF`, `sql.ErrNoRows`, `os.ErrNotExist`, `context.Canceled`, `context.DeadlineExceeded`.

Use sentinels when there's one specific failure mode and the caller might branch on it.

Typed errors

A custom struct that satisfies `error`. Use when failures carry data.

```go
type ValidationError struct {
    Field   string
    Value   any
    Message string
}

func (e *ValidationError) Error() string {
    return fmt.Sprintf("invalid %s=%v: %s", e.Field, e.Value, e.Message)
}
```

Callers extract with `errors.As`:

```go
var verr *ValidationError
if errors.As(err, &verr) {
    log.Printf("field %s failed: %s", verr.Field, verr.Message)
}
```

Examples in the stdlib: `*os.PathError`, `*url.Error`, `*net.OpError`.

Use typed errors when the failure mode has structured detail (which field, which URL, which retry attempt).

Pointer receiver vs value receiver

Almost always pointer receiver. Two reasons:

1. Comparing typed errors by identity: `errors.Is(err, target)` checks pointer equality if neither implements `Is`. Value-receiver typed errors compare by content, which can match unintended values.
2. Mutating the error (rare but sometimes useful) requires a pointer.

```
// preferred
func (e *MyError) Error() string { ... }
return &MyError{...}
```

Error wrapping (Go 1.13+)

The `errors` package added wrapping primitives so an error can carry a chain of context.

```
// at the leaf
err := doSomething()           // returns sql.ErrNoRows

// wrap as we return up
if err != nil {
    return fmt.Errorf("loading user %d: %w", id, err)
}

// further up
if err != nil {
    return fmt.Errorf("get profile: %w", err)
}

// final error message:
//   "get profile: loading user 42: sql: no rows in result set"
```

The chain is:

```
"get profile: ..." -> "loading user 42: ..." -> sql.ErrNoRows
```

errors.Unwrap

Returns the next error in the chain, or nil if there isn't one.

```
err := fmt.Errorf("wrap: %w", io.EOF)
next := errors.Unwrap(err)    // io.EOF
```

Rarely called directly. `errors.Is` and `errors.As` walk the chain for you.

errors.Is

Reports whether a target is in the error's chain.

```
if errors.Is(err, io.EOF) {
    // somewhere in the chain, an io.EOF lives
}

if errors.Is(err, sql.ErrNoRows) {
    return notFound()
}
```

Implementation: walks the chain, comparing each link to the target. A type can override the check by implementing `Is(error) bool`.

errors.As

Finds the first error in the chain that matches a target type, and assigns it.

```
var pathErr *os.PathError
if errors.As(err, &pathErr) {
    log.Printf("path %q failed: %s", pathErr.Path, pathErr.Err)
}
```

`target` must be a non-nil pointer. The function walks the chain looking for an error assignable to `*target`.

errors.Is vs errors.As

Use	When
`errors.Is(err, sentinel)`	Comparing against a known value (`io.EOF`, `sql.ErrNoRows`)
`errors.As(err, &typedTarget)`	Extracting a typed error to access its fields

Avoid `err == ErrSomething` directly. It only works if the error wasn't wrapped; `errors.Is` works either way.

errors.Join (Go 1.20+)

Combine multiple errors into one.

```go
var errs []error
for _, item := range items {
    if err := process(item); err != nil {
        errs = append(errs, fmt.Errorf("item %s: %w", item.ID, err))
    }
}
if len(errs) > 0 {
    return errors.Join(errs...)
}
```

The combined error's message is each error on its own line. `errors.Is` and `errors.As` walk all branches, not just the first.

Useful for batch operations and validation where you want to surface all failures, not just the first one.

Core handling patterns

Check immediately, return early

```go
f, err := os.Open(path)
if err != nil {
    return fmt.Errorf("open %s: %w", path, err)
}
defer f.Close()

data, err := io.ReadAll(f)
if err != nil {
    return fmt.Errorf("read %s: %w", path, err)
}
```

Each step is two lines: call, check. The error path returns; the happy path continues. Deeply nested error handling is a smell.

Wrap with context, don't replace

```go
// GOOD: chain is recoverable
return fmt.Errorf("loading config: %w", err)

// BAD: loses the original
return errors.New("loading config failed")

// ALSO BAD: silently swallows specific error info
return errors.New(err.Error())
```

The wrapped form lets the caller use `errors.Is` and `errors.As` to inspect the chain. The replaced form forces every caller to parse strings.

Add context, not noise

Each wrap layer should add information the next layer doesn't have:

```
// BAD: redundant, each layer says nothing new
"failed to load: failed to read: failed to open: ..."

// GOOD: each layer adds an identifier
"checkout: load user 42: query rows: ..."
```

The end message reads like a path through the call stack.

Don't log and return

```
// BAD: logs N times for one error, once at each layer
if err != nil {
    log.Printf("read failed: %v", err)
    return err
}
```

Pick one: either log and handle, or return for someone else to handle. Logging at every layer creates duplicate log spam for a single failure.

The rule: log at the boundary where the error stops propagating (HTTP handler, top of main, message handler). Everywhere else, return.

Don't ignore errors

```
// BAD
result, _ := doThing()

// GOOD: explicit decision
result, err := doThing()
if err != nil {
    return fmt.Errorf("do thing: %w", err)
}
```

If you genuinely want to ignore an error (e.g., closing a file in a read-only path), add a comment:

```
_ = f.Close()  // best-effort cleanup
```

Sentinel vs typed vs behavior-based

Three ways to expose error variants to callers:

1. Sentinel error: branching on identity

```
var ErrUnauthorized = errors.New("unauthorized")

// caller
if errors.Is(err, ErrUnauthorized) {
    return http.StatusUnauthorized
}
```

Use for binary checks: this error or not.

2. Typed error: extracting structured data

```
type RetryableError struct {
    Retry time.Duration
    Cause error
}
func (e *RetryableError) Error() string { ... }

// caller
var rerr *RetryableError
if errors.As(err, &rerr) {
    time.Sleep(rerr.Retry)
    goto retry
}
```

Use when the caller needs information from the error.

3. Behavior-based: interface check

Define an interface that describes a behavior, not a type:

```
type temporary interface {
    Temporary() bool
}

if t, ok := err.(temporary); ok && t.Temporary() {
    return retry()
}
```

Or with `errors.As`:

```
var t temporary
if errors.As(err, &t) && t.Temporary() {
    return retry()
}
```

This lets multiple error types signal the same trait without coupling to a single struct. The stdlib does this with `net.Error.Timeout()` and `net.Error.Temporary()` (the latter is now deprecated).

Use behavior-based when many error types might share a property (retryable, transient, idempotent).

---

panic and recover

`panic` halts execution and unwinds the stack, running deferred calls. `recover` (only inside a deferred function) stops the unwind.

```
func safe() (err error) {
    defer func() {
        if r := recover(); r != nil {
            err = fmt.Errorf("panic: %v", r)
        }
    }()
    risky()
    return nil
}
```

When to panic

Reserve `panic` for: - **Programmer bugs.** A nil deref where there shouldn't be one. An invariant that fails at startup. - **Unrecoverable state.** Required configuration is missing; the process can't continue. - **Init-time failures.** A registration that the rest of the package depends on.

Don't use panic for: - Normal failures (file missing, network error, validation). - Expected user input issues. - Anything you'd want the caller to handle.

When to recover

Almost never. Cases where it makes sense: - **Server boundary.** An HTTP handler that shouldn't let one bad request kill the process. The stdlib's `http.Server` already recovers per-handler. - **Worker boundary.** A long-running goroutine that shouldn't take down the whole program if one task panics. - **Plugin/script execution.** Code from an unknown source that you've sandboxed.

For everything else, let panic propagate. A panic that crashes the process is loud and trustworthy. A swallowed panic is a hidden bug.

Panic in goroutines

A panic in a goroutine that's not recovered crashes the entire program. The main goroutine can't recover from a child's panic.

```go
go func() {
    defer func() {
        if r := recover(); r != nil {
            log.Printf("worker panic: %v", r)
        }
    }()
    work()
}()
```

If you spawn background goroutines, recover at the top of each one. Otherwise one panic anywhere kills everything.

Panic in deferred functions

A panic during a defer overrides any previous panic.

```go
func f() {
    defer func() { panic("from defer") }()
    panic("original")
}
// final panic message: "from defer"
```

The original error is lost. Avoid panicking from deferred functions unless you know what you're doing.

Error patterns at boundaries

HTTP handler

Convert errors to status codes at the boundary, log, return a clean response.

```go
func handler(w http.ResponseWriter, r *http.Request) {
    user, err := svc.GetUser(r.Context(), id)
    if err != nil {
        switch {
        case errors.Is(err, sql.ErrNoRows):
            http.Error(w, "not found", http.StatusNotFound)
        case errors.Is(err, ErrUnauthorized):
            http.Error(w, "unauthorized", http.StatusUnauthorized)
        default:
            log.Printf("get user: %+v", err)
            http.Error(w, "internal error", http.StatusInternalServerError)
        }
        return
    }
    json.NewEncoder(w).Encode(user)
}
```

The pattern: 1. Map known errors to status codes via `errors.Is`. 2. Log unknown errors with full context (use `%+v` if your error type implements verbose formatting). 3. Return a generic message to the client; don't leak internals.

A cleaner alternative: return an error from each handler, have one middleware do the conversion.

gRPC

Convert to status codes:

```
import "google.golang.org/grpc/status"
import "google.golang.org/grpc/codes"

if errors.Is(err, sql.ErrNoRows) {
    return nil, status.Error(codes.NotFound, "user not found")
}
return nil, status.Errorf(codes.Internal, "%v", err)
```

Top of main

```
func main() {
    if err := run(); err != nil {
        fmt.Fprintf(os.Stderr, "error: %v\n", err)
        os.Exit(1)
    }
}
```

Keep `main` tiny. Move logic into `run() error`. This lets `defer` statements run before exit (which `os.Exit` skips).

Working with multiple errors

errgroup

For fan-out of parallel work with first-error semantics:

```
import "golang.org/x/sync/errgroup"

g, ctx := errgroup.WithContext(ctx)
for _, url := range urls {
    url := url
    g.Go(func() error {
        return fetch(ctx, url)
    })
}
if err := g.Wait(); err != nil {
    return err
}
```

When any goroutine errors, `ctx` is cancelled. Other goroutines see `ctx.Done()` and stop. `Wait` returns the first error.

errors.Join for collecting all

When you want every failure, not just the first:

```
var errs []error
for _, item := range items {
    if err := validate(item); err != nil {
        errs = append(errs, err)
    }
}
return errors.Join(errs...)
```

`Join` ignores nils, so you can pass partial results. `errors.Is` and `errors.As` will walk all the joined errors, not just the first.

Third-party: hashicorp/go-multierror

Predates `errors.Join`. Same idea with a richer API. Use `errors.Join` for new code.

Testing errors

```go
func TestRead_NotFound(t *testing.T) {
    _, err := Read("missing.txt")
    if !errors.Is(err, ErrNotFound) {
        t.Fatalf("got %v, want ErrNotFound", err)
    }
}

func TestParse_Validation(t *testing.T) {
    err := Parse("bad")
    var verr *ValidationError
    if !errors.As(err, &verr) {
        t.Fatalf("got %T, want *ValidationError", err)
    }
    if verr.Field != "name" {
        t.Errorf("got field %q, want name", verr.Field)
    }
}
```

Don't compare error strings:

```go
// BAD: brittle, breaks when message changes
if err.Error() != "not found" {
    t.Fatal(err)
}

// GOOD
if !errors.Is(err, ErrNotFound) {
    t.Fatal(err)
}
```

For string comparisons that are unavoidable (e.g., third-party libraries), use `strings.Contains` rather than equality.

Anti-patterns

1. Ignoring errors

```go
result, _ := doThing()       // BAD
```

If you really mean it, comment it. Usually it's a mistake.

2. Logging without returning

```
if err != nil {
    log.Println(err)
    // caller never knows it failed
}
```

The caller gets a zero value as if everything worked. Future readers can't tell the call could fail.

3. String-based error matching

```
if err.Error() == "not found" {   // BAD
    ...
}
```

The error message can change in any release. Use sentinels and `errors.Is`.

4. Wrapping with `%v` instead of `%w`

```
return fmt.Errorf("load: %v", err)    // BAD: chain broken
return fmt.Errorf("load: %w", err)    // GOOD: chain preserved
```

Use `%v` only if you specifically don't want the chain (rare).

5. Returning typed nil as error

```
func F() error {
    var e *MyError                // typed nil
    return e                      // interface wraps a non-nil type with nil data
}

err := F()
if err != nil { ... }             // TRUE, surprisingly
```

Always return concrete `nil`:

```
return nil
```

This is one of the most-cited Go gotchas. The interface value has a non-nil type pointer; `== nil` checks both pointer and type.

6. Panicking for normal failures

```
func ParseConfig(path string) Config {
    f, err := os.Open(path)
    if err != nil {
        panic(err)               // BAD: caller can't handle
    }
    ...
}
```

Return the error. Let the caller decide whether to fatal.

7. Catch-all recover

```
defer func() {
    recover()                    // hides all bugs
}()
```

Recovering without logging or rethrowing turns crashes into silent corruption. If you must recover, at minimum log the panic value and stack.

8. Error returned but not the last value

```
func bad() (error, string) { ... }     // BAD: convention is error last
```

Go convention is `error` is the last return value. Everyone reads `value, err := f()`. Breaking the convention confuses readers and tooling.

9. One-letter generic errors

```
return errors.New("error")
return errors.New("failed")
```

Useless. Include what failed and ideally why.

10. Returning errors from functions that never fail

```
func Add(a, b int) (int, error) {
    return a + b, nil            // always nil
}
```

Don’t return an error you’ll never set. Callers have to check it for nothing.

Performance considerations

Errors are cheap when there's no error. The compiler optimizes the nil-check path.

When errors do occur: - `errors.New("...")` allocates a small struct. - `fmt.Errorf` allocates for the formatted string. - `errors.Is` and `errors.As` walk the chain; cost scales with chain depth (rarely more than 3-5). - Wrapping adds about 30-50 bytes per layer.

For hot paths with frequent expected failures (e.g., a cache miss reported as error), consider: - Using a sentinel + plain comparison. - Returning a status enum instead. - Reserving the error for unexpected failures.

But profile first. Error overhead rarely dominates.

Best practices

1. **Wrap with `%w` when adding context.** Preserves the chain.
2. **Each wrap layer adds new information.** No redundant "failed to" prefixes.
3. **Check immediately, return early.** Don't pile errors into a chain.
4. **Log at the boundary, return everywhere else.** One log per failure.
5. **Sentinels for binary checks, typed for structured data, behavior interfaces for shared traits.**
6. **Pointer receivers on typed errors.** Identity-based matching.
7. **Use `errors.Is` and `errors.As` instead of `==` or string comparison.**
8. **`errors.Join` (Go 1.20+) for batch operations.**
9. **`errgroup` for parallel work with first-error cancellation.**
10. **Panic for bugs and unrecoverable startup failures only.** Everything else returns.
11. **Recover at goroutine and server boundaries** if at all.
12. **Always return concrete `nil` for success.** No typed-nil interfaces.
13. **Keep `main` thin.** A `run() error` wrapper lets defers fire.
14. **Test with `errors.Is` / `errors.As`,** not string equality.

Common gotchas

Gotcha	Symptom	Fix
Typed nil returned as `error`	`err != nil` is true unexpectedly	Return literal `nil`
`%v` instead of `%w`	`errors.Is` doesn't match the original	Use `%w`
Comparing wrapped err with `==`	Comparison always false	Use `errors.Is`
Value-receiver typed error	Unexpected matches in `errors.Is`	Use pointer receiver
Ignored error via `_`	Silent corruption	Handle or log explicitly
Logging and returning	Duplicate log spam	Pick one; log at boundary
Panic in goroutine without recover	Whole process crashes	Recover at top of each goroutine
Recover without logging	Silent bugs	Log panic + stack at minimum
Wrapping the same error twice	Garbled message	Wrap once per layer
`errors.As` with wrong target type	Returns false always	Target must be a non-nil pointer to interface/type
Sentinel error compared with `==` after wrap	Mismatch	`errors.Is`
Multiple goroutines write same error	Race / lost error	Use errgroup or channel
Logging `err.Error()`	Loses chain detail	Log `err` directly (or `%+v` if supported)
Returning error from func that can't fail	Wasted boilerplate at callers	Drop the error
`os.Exit` skips deferred cleanup	Files unclosed	Use `run() error` pattern

Quick reference

What	Code
Error interface	`type error interface { Error() string }`
Create simple	`errors.New("not found")`
Create formatted	`fmt.Errorf("loading %s: %w", path, err)`
Wrap (preserve chain)	`%w` in `fmt.Errorf`
Format without chain	`%v` in `fmt.Errorf`
Check identity	`errors.Is(err, ErrTarget)`
Extract typed	`var t *MyErr; errors.As(err, &t)`
Unwrap one layer	`errors.Unwrap(err)`
Combine many	`errors.Join(errs...)`
Sentinel pattern	`var ErrX = errors.New("...")`
Typed error pattern	`type MyErr struct { ... }; func (e *MyErr) Error() string`
Behavior interface	`interface { Temporary() bool }`
Panic	`panic("description")`
Recover	`defer func() { if r := recover(); r != nil { ... } }()`
Parallel + first error	`errgroup.WithContext(ctx)`
Status code mapping	`switch { case errors.Is(err, X): ... }`
Main pattern	`if err := run(); err != nil { os.Exit(1) }`
Test for sentinel	`if !errors.Is(err, want) { t.Fatal(err) }`
Test for typed	`var v *MyErr; if !errors.As(err, &v) { t.Fatal() }`
Goroutine safety	`defer recover` at top of every `go func()`

What	Code
Sentinel examples (stdlib)	`io.EOF`, `sql.ErrNoRows`, `context.Canceled`, `os.ErrNotExist`
Typed examples (stdlib)	`*os.PathError`, `*url.Error`, `*net.OpError`